

CHAP 10: 함수

함수 (Function) 란?
함수 (Function) 정의
함수 호출하기

학습 내용

- I. 함수란?
- II. 함수 작성하기.
- III. 함수 호출 - 함수를 호출하여 함수를 실행한다.
- IV. 인수, 매개변수
 - I. 함수에 일을 시키기 위해 넘겨주는 인수(값)를 지정하면,
 - II. 함수에게 그 값이 전달되어 매개변수에 할당된다.
- V. 여러 개의 인수를 함수로 전달 가능 - 위치 **mapping**
- VI. 함수에서 생성된 값을 반환하는 방법 - **return**문

함수(Functions)란?

- 함수(function)란?
 - 특정 작업을 수행하는 명령어들의 모음에 이름을 붙인 것.
 - 문제 해결에 필요한 작업 중 일부를 따로 떼어 구현한 것.
 - 자주 사용되는 부분 문제를 함수로 작성하고, 호출하여 재사용
- 함수는 작업에 필요한 데이터를 전달받을 수 있으며, 작업 완료 후에 결과를 함수호출자에게 반환할 수 있다.
(함수 호출 문장이 반환 값으로 대체)
- 함수를 사용하면 좋은 점
 - 프로그램 안에서 **중복되는 코드들**을 **제거**할 수 있다.
 - 작성된 함수는 **재사용(reuse)**되며, 코드 관리가 쉬워진다.



함수(Functions)란?

- 함수는 왜 필요한가?
- 예) 비슷한 코드인데 하나로 합칠 수 없을까 ? 함수를 사용

비슷한 코드인데 하나로
합칠 수 없을까?



```
s1=90 ;  
if s1 >= 90: print('A')  
elif s1 >= 80: print('B')  
elif s1 >= 70: print('C')  
else: print('D')
```

```
s2=89 ;  
if s2 >= 90: print('A')  
elif s2 >= 80: print('B')  
elif s2 >= 70: print('C')  
else: print('D')
```

```
s3=95 ;  
if s3 >= 90: print('A')  
elif s3 >= 80: print('B')  
elif s3 >= 70: print('C')  
else: print('D')
```

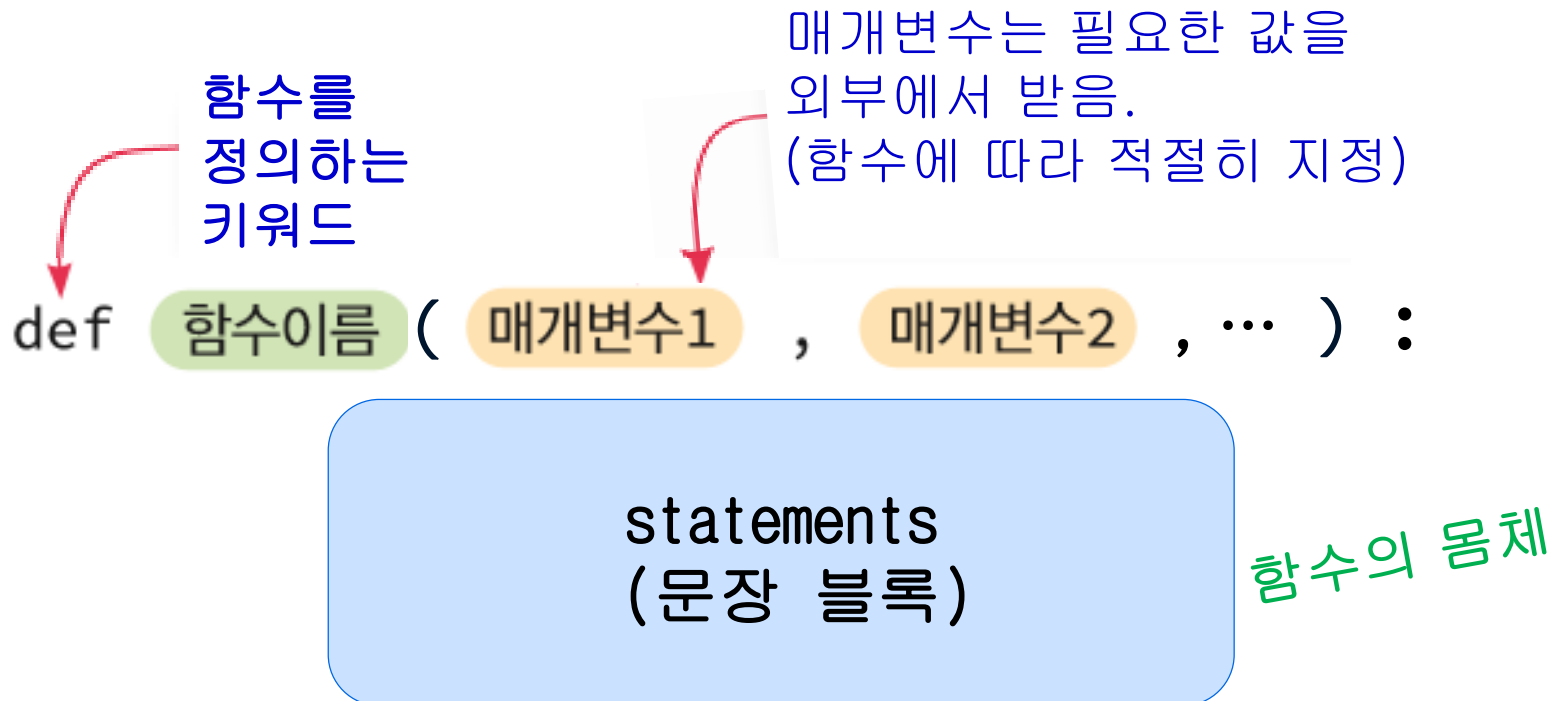
코드가
반복되어 사용



반복되는 코드를
함수로 작성하고,
함수를
호출하여 사용

함수 정의

- 함수 정의 기본 형식
 - 함수를 정의하더라도, 함수 안의 코드는 실행되지 않는다. 함수 호출을 해야 함수 안의 코드가 실행된다.
 - 함수 호출은 함수 이름을 적어주는 것.
 - 필요한 여러 곳에서 호출하여 여러 번 실행 가능.



함수 정의

- 함수 작성

```
def 함수 이름 (매개변수1, 매개변수2, ..) :  
    statements                # 함수 기능에 필요한 statements  
    return 값                 # 반환할 것이 없으면 생략 가능
```

- **def** : 함수의 시작을 알림.
- 함수 이름 : 변수명과 동일한 규칙을 따른다
- **매개변수(parameters)**
 - 함수의 입력. 콤마로 분리. 필요없으면 생략.
 - 함수 호출에서 ()안에 지정된 인수가 매개변수로 전달된다.
 - 여러 개일 경우, **위치에 따라** 인수가 매개변수로 전달
- **statements**
 - 함수의 기능을 위한 명령문들을 넣는다. **들여쓰기** 함.
- **return**
 - 값을 함수 호출한 자리로 반환. 반환 값이 없으면 생략.

함수 호출하기

- 함수를 실행하려면, **함수 호출(call)** 한다.
 - 함수를 정의하더라도, **함수 안의 코드는 실행되지 않는다.**
함수 호출을 해야 함수 안의 코드가 실행된다.
 - 함수 호출은 함수 이름을 적어 주는 것.
 - 매개변수 및 반환 값이 있는가에 따라 호출 형식이 다름.
 - 괄호 안에 지정한 인수 값은 매개변수로 전달.
 - **인수(argument)**는 위치에 따라 매개변수로 전달됨.

변수 = 함수명(인수1, 인수2, ...)

#매개변수, 반환 값 있는 경우.
#반환 값이 변수에 할당됨

변수 = 함수명()

#매개변수 없고, 반환값 있는 경우.
#반환 값을 변수에 할당

함수명(인수1,인수2,...)

#매개변수 있고, 반환 값 없는 경우

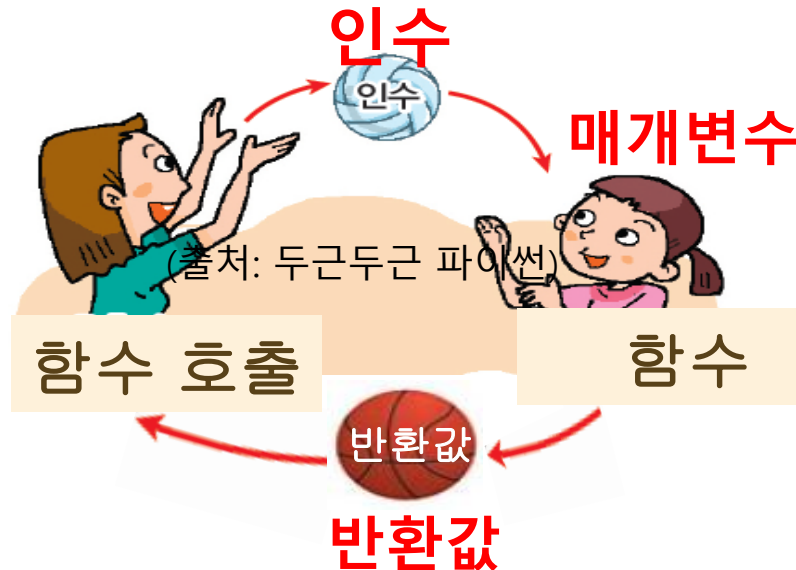
함수명()

#매개변수, 반환 값 모두 없는 경우

함수 호출하기

- 정의된 함수는 함수 호출 해야 실행된다.
- 인수와 매개변수는 함수 호출 시에 데이터를 전달에 사용.
함수 호출에서 지정된 인수는 함수의 매개변수로 전달된다.
 - 인수(argument) - 함수호출 때에 지정. 함수에게 전달되는 값
 - 매개 변수(parameter)- 인수를 전달받는 변수. (참조값을 전달받음)

변수 = 함수명(인수1, 인수2, ...) # 함수 호출



```
def 함수 (매개변수) :  
    -----  
    -----  
    -----  
    return 값
```


함수 호출하기

- 인수(argument) – 호출 때에 지정하여 함수에 전달되는 값
- 매개 변수(parameter)– 인수를 전달받는 변수. (참조값이 전달)
- 여러 개 일 경우, 위치에 따라 인수가 매개변수로 전달된다.

예) Grade를 판별 / 출력 함수

함수 호출

gPrint(90)

인수

gPrint(s1)

인수

매개변수

함수

```
def gPrint( score ) :  
    if score >= 90:    print('A')  
    elif score >= 80:  print('B')  
    elif score >= 70:  print('C')  
    else:              print('D')
```

함수 호출하기

- 예) grade 판별하고 출력하는 함수
 - 반복되는 코드를 함수로 정의하고, 함수 호출!

```
s1=90 ;  
if s1 >= 90: print('A')  
elif s1 >= 80: print('B')  
elif s1 >= 70: print('C')  
else: print('D')
```

```
s2=89 ;  
if s2 >= 90: print('A')  
elif s2 >= 80: print('B')  
elif s2 >= 70: print('C')  
else: print('D')
```

```
s3=95 ;  
if s3 >= 90: print('A')  
elif s3 >= 80: print('B')  
elif s3 >= 70: print('C')  
else: print('D')
```



시작

```
def gPrint(score) :  
    if score >= 90: print('A')  
    elif score >= 80: print('B')  
    elif score >= 70: print('C')  
    else: print('D')
```

함수 정의

```
s1=90  
gPrint(s1) #함수 호출
```

```
s2=89  
gPrint(s2) #함수 호출
```

```
s3=95  
gPrint(s3) #함수 호출
```

함수 호출하기

```
def gPrint(score) :
```

```
    if score >= 90:    print('A')
    elif score >= 80:  print('B')
    elif score >= 70:  print('C')
    else:               print('D')
```

함수 정의

시작

```
#main
```

```
s1=90
gPrint(s1)
```

```
s2=89
gPrint(s2)
```

```
s3=95
gPrint(s3)
```

90

score 매개변수

gPrint() 함수

반환값 X

```
s1=90 ;
if s1 >= 90:    print('A')
elif s1 >= 80:  print('B')
elif s1 >= 70:  print('C')
else:           print('D')
```

```
s2=89 ;
if s2 >= 90:    print('A')
elif s2 >= 80:  print('B')
elif s2 >= 70:  print('C')
else:           print('D')
```

```
s3=95 ;
if s3 >= 90:    print('A')
elif s3 >= 80:  print('B')
elif s3 >= 70:  print('C')
else:           print('D')
```

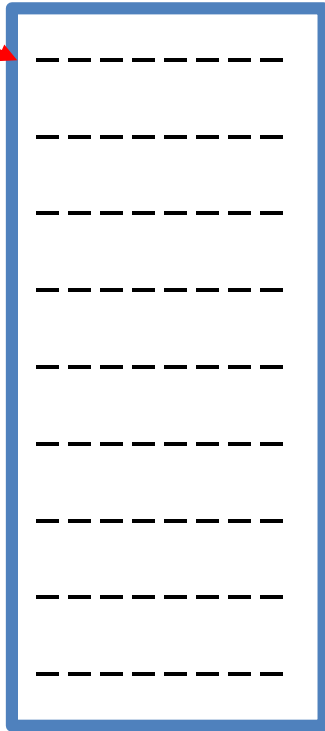
😊 출력 화면

A
B
A

함수 호출하기

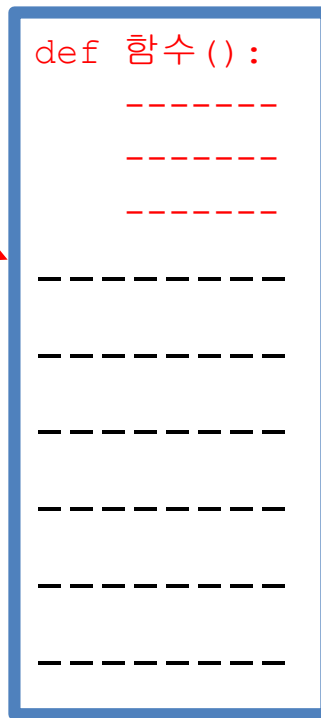
- 프로그램에서 코드가 실행되는 첫 부분을 **main**코드라고 함.
- 파이썬은 실행을 시작하는 첫 부분(**main 함수**)이 따로 없다.
 - 대신 **들여쓰기 하지 않은 첫 문장부터 실행**한다.
들여쓰기 하지 않은 모든 코드가 **main** 함수라 할 수 있다.

시작



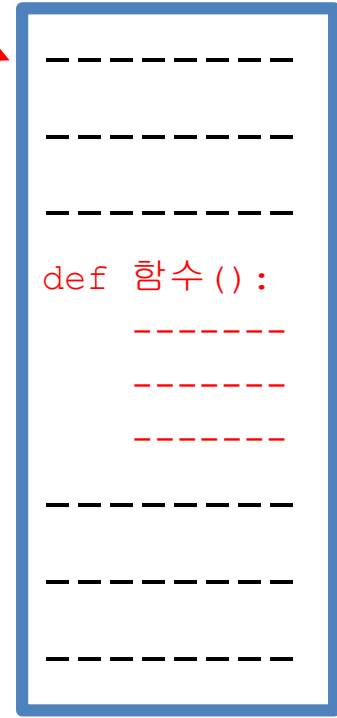
함수가 없는 코드

시작



함수가 맨 앞에
있는 코드

시작



함수가 중간에
있는 코드

함수 호출하기

- 함수와 반복문의 차이??

- 함수를 사용하면, 프로그램의 여러 곳에서 동일한 작업이 가능.
- 함수에 값을 전달하여서 호출 할 때마다 다르게 실행할 수 있다.

비교

```
student=[90, 89,95]
```

```
for score in student:
```

```
    if score >= 90:    print('A')
    elif score >= 80: print('B')
    elif score >= 70: print('C')
    else: print('D')
```

😊 출력 화면

A
B
A

```
def gPrint(score) :
    if score >= 90:    print('A')
    elif score >= 80: print('B')
    elif score >= 70: print('C')
    else: print('D')
```

```
s1=90
gPrint(s1)
```

```
s2=89
gPrint(s2)
```

```
s3=95
gPrint(s3)
```

함수 호출하기

- 예) grade 판별 함수
 - 매개변수 사용. grade를 판별하고 반환한다.

```
def gChk( score ) :  
    if score >= 90: grade= 'A'  
    elif score >= 80: grade= 'B'  
    elif score >= 70: grade= 'C'  
    else : grade= 'D'  
    return grade
```

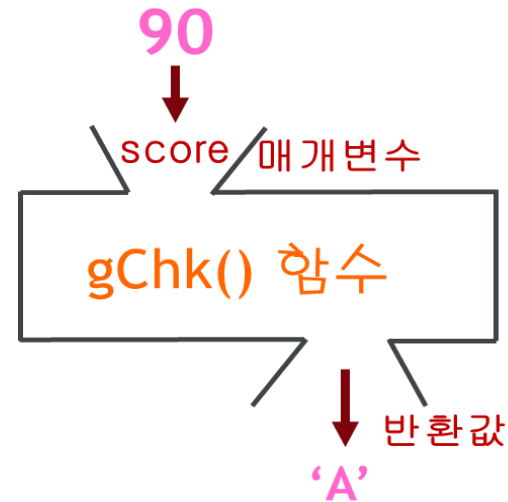
main part

국어 = 90

```
g1= gChk(국어) #함수호출  
print(g1)
```

수학 = 89

```
g2= gChk(수학) #함수호출  
print(g2)
```



😊 실행 화면

A
B

함수 호출하기

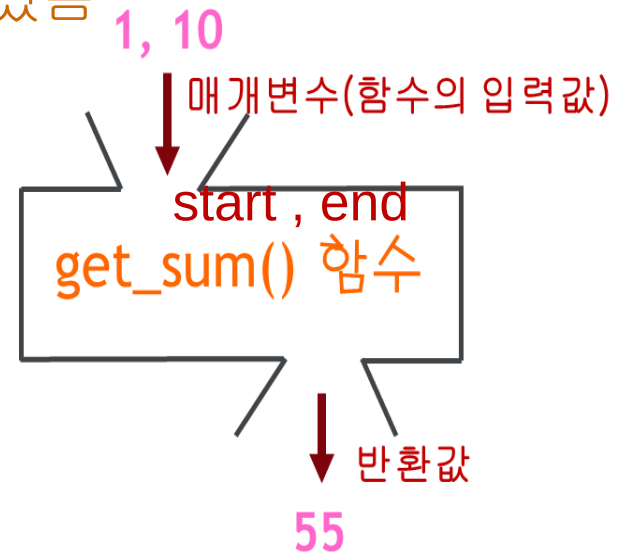
- 예) 합계 구하기 함수. 매개변수 2개, 반환 값 있음

- 매개변수 개수 만큼 인수를 지정,
위치에 따라 인수가 매개변수로 전달된다.

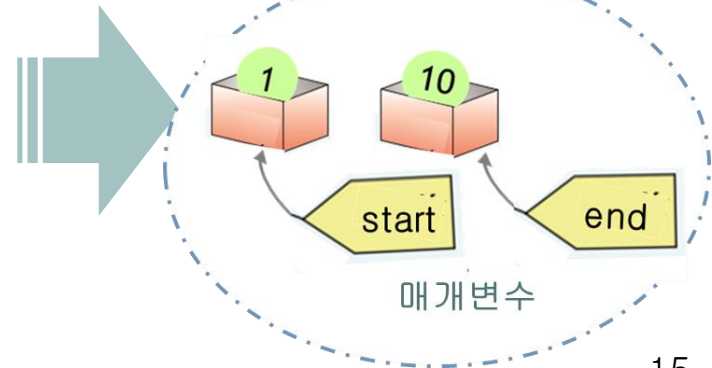
```
def get_sum(start, end):  
    hap=0  
    for i in range(start, end+1):  
        hap = hap+i  
    return hap # 반환 값
```

#main part

```
hap1= get_sum(1, 10) #함수호출  
print('합', hap1) # 반환 값 출력
```



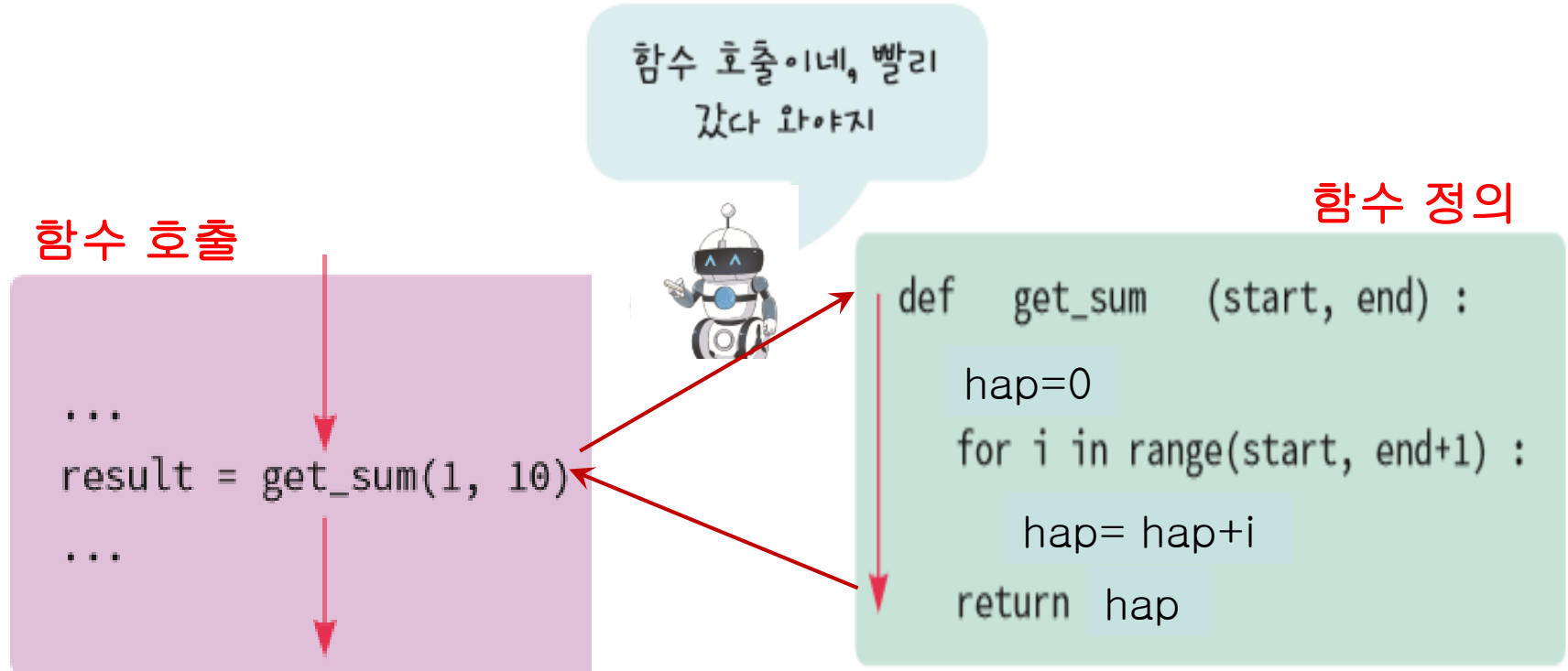
호출시 지정한 인수가
매개변수에 전달.



함수 호출하기

- 함수가 호출되면 실행의 흐름은 어떻게 될까?
 - 함수가 호출되는 시점에 실행의 흐름은 함수로 넘어간다.
함수의 작업이 끝나면 실행의 흐름은 다시 호출된 곳으로 돌아온다.

(출처: 어서와 파이썬은 처음이지)



함수 호출하기

- 함수의 위치
 - 함수는 호출 전에 정의되어 있어야 한다.
 - 함수 호출 전에 정의되어 있지 않으면 에러 발생

```
print('start of the program')
hello() #오류. NameError 발생
print('middle of the program')
hello()
print('end of the program')

def hello(): # 함수 정의
    print('hello world')
```

여기에서
프로그램 실행
시작 →

```
def hello() :
    print('hello')
    print('world')

# main - 여기에서 프로그램 실행 시작
print('start of the program')
hello() # 함수 호출
print('middle of the program')
hello() # 함수 호출
print('end of the program')
```

😊 실행 화면

```
Traceback (most recent call last):
  File "<pyshell#0>", line 1, in <module>
    hello()
NameError: name 'hello' is not defined
```

함수 호출하기

- 매개변수 개수 만큼 인수를 지정하여 함수를 호출해야 함.
위치에 따라 인수가 매개변수로 전달된다.
- (예) 함수 호출에서 인수를 2개 지정한다.

```
def nPrintln(message, n) : #매개변수가 2개
    for i in range(0, n) :
        print(message)
```

nPrintln(" 사랑한다 사랑한다 사랑한다 " , 3)

😊 실행 화면

사랑한다 사랑한다 사랑한다
사랑한다 사랑한다 사랑한다
사랑한다 사랑한다 사랑한다

- 다음 구문은 무엇이 잘못?
nPrintln('사랑한다 사랑한다 사랑한다') # error. 개수 불일치
nPrintln(4) # error. 개수 불일치
nPrintln() # error

모듈 사용: import

- import 의 역할

- “다른 모듈 내의 코드에 대한 접근”을 가능하게 하는 것
- “다른 모듈”에는 변수, 함수 등이 포함되어 있다

- import의 사용 방법

```
import 모듈이름
```

모듈내 모든 함수 사용가능

```
from 모듈이름 import *
```

모듈내 모든 함수 사용가능

- **모듈이름**: 모듈의 실제 파일명은 “**모듈이름.py**”
- * (와일드카드)는 모든 것을 의미. 모듈 내의 모든 함수를 불러옴.

모듈 사용: import

① `import 모듈이름`

모듈내 모든 함수 사용가능

- 모듈내 모든 함수 사용가능
- `import` 후 모듈내의 함수 사용은 ? `모듈이름.함수이름()`

② `from 모듈이름 import *`

모듈내 모든 함수 사용가능

- 모듈내 모든 함수 사용가능
- `import` 후 모듈내의 함수 사용은 ? `함수이름()`